Sorted Pagination of Authorized Rows in Hierarchical Role-Based Access Control

Ross Slaney

***Abstract*—A company that organizes its stores into regions and chains organizes responsibility the same way: a regional manager answers for every store in the region, a store manager for one store. Role-based access control with scopes expresses this directly. A role is a set of permissions, and assigning it to a person at a resource makes the permissions apply to that resource and to everything beneath it, so the same role assigned at a region or at a store covers a different set of products. Azure role-based access control and several authorization services for application developers use this model, and SHRBAC enforces it inside an application's own relational database, where an access check added to a SQL query decides each row. Checking one row is fast, but listing the rows a person may see, sorted and one page at a time, is not. The database reads rows in sort order and checks each one until a page is full, so a store manager's first page of 20 products sorted by price reads 184,205 of 1,000,000 products on PostgreSQL 16, and a manager whose 20,000 products have the highest prices reads 980,020. Two separate pieces of information determine such a page. The requested permission and the person's role assignments determine a small set of resources whose products are the person's, and the sort index supplies the order, but for all products at once. We keep them separable. An ancestor-partitioned index projection stores every product once under each resource above it, in index order, so the products beneath any resource form one sorted range. A page reads the first entries of the range of each resource where the person holds the permission, merges them, and re-checks the selected rows. We prove that the method returns only authorized rows whatever the projections contain, and all of them while the projections are current, and we bound the cost of a page by the page size, the hierarchy depth and the person's assignments rather than by the table size. On SQL Server 2022 and PostgreSQL 16 the two first pages above read 100 and 80 rows. The costs are one projection entry per row and ancestor for each index, maintained on every write, and one range read per assignment for people who hold thousands of separate assignments.**

I. Introduction

A business application has to answer, for every request, whether the person making it may see or change the data involved. In a retailer, the answer usually follows the shape of the company. Products belong to stores, stores to regions, regions to chains, and chains to divisions. A regional manager is responsible for every store in the region and should be able to view and reprice every product in those stores. A store manager is responsible for one store. An auditor may view products across a region but change nothing. These three people hold different authority over different parts of the same hierarchy, and the application must enforce exactly that.

Role-based access control with scopes expresses such authority with three pieces. A permission names an operation on a kind of data, such as *view products* or *change prices*. A role is a named set of permissions: a Manager role might carry both, and a Viewer role only the first. An assignment gives a role to a person at a resource in the hierarchy, and the role's permissions then apply to that resource and to every resource beneath it. The set of resources an assignment reaches is its scope. Maria, assigned Manager at region R1, may view and reprice every product in every store of R1. Lee, assigned Manager at store S2 and Viewer at R1, may view products across the region but may change prices only in S2. The two hold the same roles; what differs is where the roles were assigned.

This model is in wide commercial use. Azure role-based access control (Azure RBAC) assigns a role definition, a collection of permissions, to a security principal at a scope, which is a management group, a subscription, a resource group or a single resource, and scopes form a hierarchy in which lower levels inherit the assignments made at higher levels, so an assignment at a resource group applies to every resource in it [1], [2]. Authorization services for application developers provide the same inheritance over resources that the application defines. WorkOS arranges an application's resources in a hierarchy down which permissions flow [3]; OpenFGA lets a relation on a folder, such as *editor*, apply to the documents the folder contains [4]; and Google's Zanzibar expresses inheritance from a parent object through userset rewrite rules [5]. Classic role-based access control assigns permissions to roles and roles to users [6], [7]; the scope is what makes the model hierarchical.

SHRBAC [8] enforces this model inside the application's own relational database. The resource hierarchy, the roles and the assignments, which SHRBAC calls grants, are tables stored next to the application's data, and every protected row, such as a product, records the resource it belongs to. Deciding whether Maria may view product p3 in store S2 is a walk up the hierarchy from S2: is there a grant to Maria at S2 whose role carries *view products*? No. At R1, S2's parent? Yes, Maria holds Manager at R1, and Manager carries the permission, so the answer is yes. Deciding whether Lee may reprice product p1 in store S1 walks from S1 to R1 to the chain, the division and the root, finds only Lee's Viewer grant at R1, which does not carry *change prices*, and answers no. SHRBAC implements this walk as a SQL function that an application adds to its own queries, so a single SQL statement can filter, sort and page the rows a person may see. The concepts are those of scoped role-based access control; the contribution of [8] is their enforcement inside the database over application-defined resources, with proofs that the check is sound and complete and a bound on its cost: a check visits

at most D ancestors, the depth of the hierarchy, for each of the person's M principals, which are the person and the person's groups. On a hierarchy of depth 9 with 1.5 million resources, a single check took under 1.5 ms (median).

Checking one row and listing many rows are different problems. Consider a store manager who opens the product list of the store, sorted by price, 20 products per page. The products that qualify are the store's 100 products. The table, however, holds 1,000,000 products from 10,000 stores, and the index on price that sorts them interleaves the store's products with everyone else's. To produce the first page, the database walks the index in price order and applies the access check to each product until 20 have passed. Every product of another store that is cheaper than the store's 20th cheapest product is read and rejected on the way. On PostgreSQL 16 the first page reads 184,205 products (Section VII). The number read depends on where the person's products fall in the order, not on the cost of a check. A person who may see a fraction σ of the rows, spread evenly, reads about P/σ rows for a page of P; a person whose V rows come last reads $N-V+P$ rows for the first page, which is 980,020 for a chain manager whose 20,000 products have the highest prices. The earlier paper states this dependence in its cost analysis and evaluated people whose rows are dense in the order [8]. Authorization services limit the same operation: OpenFGA's ListObjects returns at most 1,000 results within 3 seconds by default [9], and SpiceDB's documentation recommends its LookupResources call when a user can access fewer than about 10,000 resources [10].

Two separate pieces of information determine the page. The first is which resources authorize the person for the requested permission: for Maria and *view products* it is region R1; for Lee it is R1 as well, because the Viewer grant at R1 already covers store S2; for Lee and *change prices* it is S2 alone. This set is computed from the grant tables, without reading any product, and it is small for most people. The second is the order of the products, which the index on price provides, but for all products of the table together. Checking rows in sort order takes the order from the index and consults the grants once per row; collecting the person's rows first takes membership from the grants and then sorts. Neither has what a page needs, namely the rows of each authorizing resource, in index order. This paper adds that structure. An ancestor-partitioned index projection stores, for one index of a table, every row once under each resource above it, sorted by the index key, so the rows beneath any resource form one sorted range. Lee's first page for *view products* is then the first 20 entries under R1. A person whose permission is authorized at several resources reads the first entries under each of them and merges these sorted lists. Every selected row is re-checked with the original access function, so the projections decide which rows are found but never which rows are allowed, and they are updated in the same transaction as every write to the data they summarize.

This paper makes four contributions.

- An analysis of sorted pagination under scoped role-based access control. Checking rows in sort order reads every row up to the P-th one the person may see, and collecting the person's rows through an ancestor closure, a table that pairs every row with every resource above it, reads all of them. For some people each quantity grows with the size of the table, and a query optimizer cannot choose between the two methods because neither quantity is a property of the table (Section III).
- Ancestor-partitioned index projections, together with a page algorithm in three steps, a short scan in sort order, a merge of the person's ranges and a final access check, with the resources that authorize the person computed inside the database (Section IV).
- Proofs that the algorithm returns only rows the person may see, whatever the projections contain, and misses none while the projections are current, and a bound on the cost of a page, $O(P \cdot D \cdot M \cdot G_{\max} + G \cdot D + k \cdot P \log k)$ operations for a person with G relevant grants that reduce to k resources, which does not depend on the size of the table (Section V).
- An implementation for SQL Server and PostgreSQL that derives the projections from the application's existing index declarations and maintains them in the writing transaction (Section VI), and an evaluation with 1,000,000 products that compares it with both alternatives for people whose products are common, rare, late in the order or spread over thousands of separate grants (Section VII).

Section VIII relates the method to authorization services, row-level security and hierarchy encodings, and Section IX states its limitations.

II. Scoped Roles over a Resource Hierarchy

A. The Model by Example

Fig. 2(a) shows the part of the retailer's hierarchy that the rest of the paper uses. The root holds division D1, D1 holds chain C1, C1 holds regions R1 and R2, R1 holds stores S1 and S2, and R2 holds store S3. Six products are shown: p1 and p2 in S1, p3 and p4 in S2, p5 and p6 in S3. Each product has a price, and the Products table (Fig. 2(b)) records each product's identifier, price and the resource it belongs to.

Table I lists the people of the example and their grants. A grant has three parts: a principal, a role and the resource at which the role applies. Maria's single grant, Manager at R1, lets her view and reprice p1 through p4, the products of the two stores in R1, and nothing in S3. Lee's two grants give him *view products* over all of R1, through the Viewer grant, and *change prices* only in S2, through the Manager grant. Sam, an auditor, holds Viewer at two stores in different regions, S2 and S3, so Sam may view p3 through p6 and may change nothing.

Three further rules complete the model. First, people can belong to groups, and a grant to a group applies to its members; if the three store managers of R1 belong to a group that holds Viewer at R1, each of them may view the whole region. Groups do not contain other groups. Second, a grant may carry a validity interval, such as a temporary assignment that ends on a fixed date, and only grants active at the time

TABLE I
PEOPLE, ROLES AND GRANTS OF THE RUNNING EXAMPLE. THE MANAGER ROLE CARRIES THE PERMISSIONS VIEW PRODUCTS AND CHANGE PRICES; THE VIEWER ROLE CARRIES VIEW PRODUCTS.

Person	Grant (role at resource)	May view	May reprice
Maria	Manager at R1	p1, p2, p3, p4	p1, p2, p3, p4
Lee	Viewer at R1; Manager at S2	p1, p2, p3, p4	p3, p4
Sam	Viewer at S2; Viewer at S3	p3, p4, p5, p6	none

of the request count. Third, a resource can be deactivated, for example when a store closes, and an inactive resource stops inheritance: a grant at R1 no longer reaches the products of a closed store in R1. A permission may also be restricted to one type of resource, so that *change prices* applies to products but not to the stores above them.

B. The Same Model in Azure RBAC and Other Platforms

The structure of Section II-A is the structure of Azure RBAC, and Table II sets the two side by side. In Azure, a role assignment consists of a security principal (a user, group, service principal or managed identity), a role definition and a scope [1], [11]. A role definition lists the operations the role permits, its Actions and DataActions, and operations excluded from them, its NotActions and NotDataActions; an exclusion is not a denial, because another assignment may grant the excluded operation [12]. Scopes have four levels, management groups, subscriptions, resource groups and resources, and lower levels inherit the role permissions assigned at higher levels [2]. Management groups form a tree of up to six levels below the root, and each management group and subscription has only one parent [13]. A principal's effective permissions are the sum of its role assignments [1]. Group membership is transitive, so a user in a group that belongs to an assigned group receives the assignment [1].

Azure adds features that SHRBAC omits or simplifies. Deny assignments block actions even when a role grants them, and in generally available scenarios Azure creates them rather than customers [14]. Role assignments may carry attribute conditions [11], and Privileged Identity Management assigns roles with start and end dates [15]. The place of enforcement also differs. To decide a request, Azure Resource Manager collects the role and deny assignments that apply to the resource, blocks the request if a deny assignment applies, determines the roles the caller and its groups hold, checks whether one of them permits the action, and evaluates any conditions [1]. SHRBAC evaluates the equivalent decision as a predicate inside the application's own database queries.

Earlier models in the research literature scope role assignments in the same way: ROBAC attaches assignments to the nodes of an organization hierarchy [16], and RBAC with resource hierarchies to the nodes of a resource hierarchy [17]; [8] relates SHRBAC to these models and to RBAC96 and the NIST RBAC standard.

C. Definitions

The definitions below restate the model of [8] in the terms used by the rest of the paper.

Resources form a rooted tree of depth at most D, and each resource may be marked inactive. For a resource r, $\mathrm{anc}(r)$ is the sequence $r, \mathrm{parent}(r), \mathrm{parent}(\mathrm{parent}(r)), \ldots$ up to the root; it includes r itself. The active ancestor path $\mathrm{anc}^{+}(r)$ is the longest prefix of $\mathrm{anc}(r)$ in which every resource is active, and it is empty when r is inactive. In Fig. 2, $\mathrm{anc}^{+}(\mathrm{S2}) = (\mathrm{S2}, \mathrm{R1}, \mathrm{C1}, \mathrm{D1}, \mathrm{root})$ while every resource is active.

Principals are users, groups, service accounts and agents. For a principal p, $\mathrm{resolve}(p)$ contains p and, if p is a user, the active groups of which p is a member. We write $M = |\mathrm{resolve}(p)|$ for the number of principals whose grants count for p. A role is a set of permissions, written $\mathrm{perms}(\mathit{role})$, and a permission may be restricted to one resource type. A grant $(p, \mathit{role}, x, t_1, t_2)$ assigns a role to principal p at resource x for the interval $[t_1, t_2]$, either bound of which may be absent; it is active at time t if t lies in the interval.

Definition 1 (Access decision). For a principal p, permission perm, resource r and time t, $\mathrm{allowed}(p, \mathit{perm}, r, t)$ holds if and only if there exist a principal $p' \in \mathrm{resolve}(p)$, a resource $x \in \mathrm{anc}^{+}(r)$ and a grant $(p', \mathit{role}, x, t_1, t_2)$ active at t with $\mathit{perm} \in \mathrm{perms}(\mathit{role})$, and, if perm is restricted to a resource type, r has that type.

Definition 1 is the walk of Section I written as a condition: some principal of the person holds, at some active ancestor of the resource, an active grant whose role carries the permission. An application table takes part in authorization when every row names a resource in a column ResourceId, as the Products table does. We call such a table protected and write $\mathrm{res}(r)$ for the resource of row r. A row r is authorized for p and perm at t if $\mathrm{allowed}(p, \mathit{perm}, \mathrm{res}(r), t)$ holds. In the example, every product names its store; in the benchmark of Section VII, every product is a resource of its own below its store, which adds one level to the walk.

D. Enforcement Inside the Database

In SHRBAC the access decision is a SQL function, and the application uses it by adding it to its queries. The function, `fn_IsResourceAccessible`, which we call the access function, takes a resource identifier, the identifiers of the caller's principals (the person and the person's groups) and a permission identifier. A recursive common table expression walks from the resource toward the root for at most D steps and joins each step with the active grants of the given principals whose roles carry the permission; the function returns a row as soon as one such grant is found. The function is inline, which means the query optimizer substitutes its body into the calling query rather than invoking it separately for every row, so the walk is planned and executed as part of the query's own plan. A query keeps only the rows the caller may see by adding the condition `EXISTS (SELECT 1 FROM fn_IsResourceAccessible(T.ResourceId, ...))`.

The function is sound and complete with respect to Definition 1 for hierarchies within the depth bound, and one

TABLE II
AZURE RBAC AND SHRBAC.

Concept	Azure RBAC	SHRBAC
Principals	users, groups, service principals, managed identities	users, groups, service accounts, agents
Role	role definition: Actions and DataActions, less NotActions and NotDataActions	set of permissions
Scope hierarchy	management groups (up to six levels below the root), subscriptions, resource groups, resources; one parent per node	application-defined tree of typed resources of depth at most D; one parent per node
Assignment	role assignment: principal, role definition, scope	grant: principal, role, resource, validity interval
Inheritance	lower levels inherit role permissions from higher levels	a grant applies at its resource and at every descendant reached through active resources
Multiple assignments	additive	additive
Group membership	transitive	one level; groups do not contain groups
Negative authorization	deny assignments, created by Azure	none
Other conditions	attribute conditions; start and end dates through Privileged Identity Management	validity interval of each grant
Enforcement	evaluated for each request by Azure Resource Manager or integrated data-plane services	evaluated by the application's database as a predicate within each query

evaluation costs $O(D \cdot M \cdot G_{\max})$ index lookups, where $G_{\max}$ is the largest number of active grants that one principal holds at one resource [8]. For a page of P rows requested with cursor pagination on an indexed sort column, [8] gives $O(P \cdot D \cdot M \cdot G_{\max})$ when nearly all rows are authorized, $O(P/\sigma \cdot D \cdot M \cdot G_{\max})$ when a fraction σ of them is, and $O(N \cdot D \cdot M \cdot G_{\max})$ when none is. These three bounds are the subject of Section III. The method of Section IV uses the function unchanged in its first and last steps.

Notation. N is the number of rows of the paged table and P the page size. D is the depth bound of the hierarchy, and $L(r) = |\mathrm{anc}^{+}(\mathrm{res}(r))| \leq D + 1$ the number of active ancestors of a row's resource, including the resource itself. For a given caller and permission, V is the number of authorized rows, $\sigma = V/N$ the caller's selectivity, G the number of active grants held by the caller's principals whose roles carry the permission, and k the number of minimal granted scopes (Section IV-C). R is the number of rows read by checking rows in sort order (Section III-B), and I the number of projected orderings of a table (Section IV-B). As in [8], an index seek counts as one operation; with B-tree indexes a seek costs $O(\log N)$ page accesses.

III. LISTING AUTHORIZED ROWS IN SORTED PAGES

A. What a Page Request Asks For

A list screen shows the first P rows that the person may see, in a chosen order, and a request for the next page continues after the last row shown. Let T be a protected table with primary key κ. An ordering $K = (c_1, \ldots, c_m, \kappa)$ is a sequence of columns of T that ends with the primary key, so that rows with equal values in the other columns still have a fixed order; K therefore orders the rows of T totally, and we write $K(r)$ for the values of K in row r. A paged query $Q = (T, \varphi, K)$ restricts T by an application-defined filter φ, such as a match on the product name, and orders it by K, ascending or descending. A cursor u is either empty, for the first page, or the values of K in the last row of the previous page.

Definition 2 (Authorized page). For a caller p, permission $perm$, time t, cursor u and page size P, the authorized page of Q is the sequence of the first P rows r of T in K order such that $K(r)$ follows u, $\varphi(r)$ holds, and r is authorized for p and $perm$ at t. If fewer than P rows qualify, the page contains all of them.

For Lee and *view products*, sorted by price with $P = 2$, the first page is p1 (price 3) and p3 (price 5): p5 and p6 are cheaper but belong to S3, where Lee holds nothing. The cursor after this page is the price and identifier of p3, and the next page is p4 and p2.

Continuing from a cursor rather than from a page number is keyset pagination. The database seeks to the cursor's position in the index instead of counting past the rows of earlier pages, so the cost of a page does not depend on how many pages precede it [8]; the dependence studied below is on the rows between the cursor and the end of the page.

A solution has to respect the setting in which the method is used. The application owns T and its schema, so a method may add the ResourceId column and an index on it, and nothing else, to the application's tables; any other structure lives in separate tables. Every authorization decision stays inside the database: a statement receives the caller's principal identifiers and the permission identifier, and any set derived from the caller's grants, such as the resources at which the caller holds the permission, is computed inside the statement, because its size is not bounded. Derived structures are updated in the transaction of each write that changes them, so the application never observes a window in which they disagree with the data. Every returned row satisfies Definition 1 when the statement executes. Within these conditions, the goal is a cost per page that does not depend on N or on the caller's selectivity.

B. Checking Each Row in Sort Order

The method of [8] reads the rows of Q after the cursor through the index on K and evaluates the access function on each one until P have passed. In the example of Fig. 2, Lee's first page with $P = 2$ reads p5 and p6, rejects both, then reads and accepts p1 and p3: four rows for a page of two. Let $R(u)$ be the number of rows of Q after u up to and including the P-th authorized row, or all rows of Q after u if fewer than P

are authorized. The page costs $\Theta(R(u))$ row reads and $R(u)$ evaluations of the access function, each $O(D \cdot M \cdot G_{\max})$.

$R(u)$ depends on where the caller's rows fall in K order. If the caller may see a fraction σ of the rows of Q and those rows are spread evenly through the order, about one row in $1/\sigma$ passes, and $R(u)$ is about P/σ. If the caller's V rows are the last V rows of Q, every other row of the table precedes them, and the first page has $R = N - V + P$. Fig. 1 shows both cases in the workload of Section VII, a table of 1,000,000 products: a store manager with 100 products spread through the price order has $R = 184{,}205$, and a chain manager whose 20,000 products have the highest prices has $R = 980{,}020$. Making each check faster, for instance with an index on ResourceId, does not change R, because the method still has to look at every row that precedes the page.

C. Collecting the Caller's Rows First

The dependence on position disappears if the caller's rows can be found without looking at anyone else's. An ancestor closure of T is a table that pairs every row with every active ancestor of its resource, the set of pairs (x, r) with $r \in T$ and $x \in \mathrm{anc}^+(\mathrm{res}(r))$, indexed by x. Looking up R1 in the closure of the example returns p1, p2, p3 and p4 directly, and looking up a region of the benchmark returns its 2,000 products with one index seek. The caller's authorized rows are the rows paired with the resources where the caller holds a qualifying grant.

The closure does not sort them. The pairs under R1 are found by resource, not by price, so returning the first P in K order means reading every authorized row after the cursor that satisfies φ, sorting them, and keeping the first P. The first page costs $\Theta(V)$ row reads: the 20 cheapest of a chain manager's 20,000 products are found only by reading all 20,000. Position no longer matters, but the number of rows the caller may see does.

Running both methods in rounds with growing budgets, and taking the page from whichever finishes first, reads $O(\min(R, V) + P)$ rows. This still grows with the table for some callers. For the chain manager, $V = 0.02N$ and $R = N - V + P$, so both terms grow in proportion to N when the manager's share of the table is fixed. Choosing one method in advance would require knowing R and V, and both depend on the caller's grants. A query optimizer chooses a plan from statistics about the table, such as how prices are distributed, and those statistics are the same for every caller; they cannot tell the optimizer whether this caller's rows come early or late in the order, or how many there are.

D. Two Pieces of Information

A page depends on two things that come from different places. The first is which resources authorize the caller for the requested permission. For Maria and *view products* it is R1; for Lee it is also R1, because the Viewer grant at R1 already covers S2; for Lee and *change prices* it is S2 alone; and for Sam and *view products* it is S2 and S3. This set is computed from the grant and resource tables, without reading any product, and the rows the caller may see are exactly the rows beneath these resources. The second is the order of the rows, which the index on the sort column provides, but for all rows of the table together, since an index knows nothing about resources.

Each method of this section uses one piece and pays for the other. Checking rows in sort order takes the order from the index and must consult the grants for every row it passes; collecting through the closure takes membership from the grants and must sort what it collects. A page could be assembled cheaply if, for each authorizing resource, the rows beneath it were available already sorted by K. The resources would come from the grants, the sorted rows beneath each resource from the structure, and the page from the first few rows of each. Section IV builds that structure by partitioning each index by ancestor.

IV. Ancestor-Partitioned Index Projections

A. What a Projection Stores

A projection takes one index of a protected table and lists every row once under each active ancestor of its resource, sorted by the index key within each ancestor. Fig. 2(c) shows the projection of the Products table on the ordering (Price, Id). Under S1 it lists p1 and p2 in price order, under S2 p3 and p4, and under S3 p5 and p6. Under R1 it lists all four products of S1 and S2, again in price order, and under R2 the two products of S3. Under C1, D1 and the root it lists all six. Each product therefore appears once per ancestor, five times in this example, and the entries under any one resource, read in order, are the products beneath that resource sorted by price. Maria's products are the four consecutive entries under R1.

Definition 3 (Projection). For a protected table T and an ordering K, the ancestor-partitioned index projection of T on K is the relation

$$\Pi_K(T) = \{\, (x, K(r)) : r \in T,\ x \in \mathrm{anc}^+(\mathrm{res}(r)) \,\}.$$

It is stored in a separate table with primary key $(x, c_1, \ldots, c_m, \kappa)$, which keeps the entries under one resource together and in K order, and a secondary index on κ, which finds every entry of a row when the row changes. A stored projection is consistent with T and the resource tables if it contains exactly these entries.

A row has an entry under exactly the resources at which a grant could authorize it: by Definition 1, a grant at x authorizes row r only if $x \in \mathrm{anc}^+(\mathrm{res}(r))$, which is the membership condition of Definition 3. Each row has $L(r) \le D + 1$ entries in each projection, and a row whose resource is inactive has none.

B. Which Orderings Get a Projection

An application sorts its lists by columns it has indexed, and the method keeps one projection per index, so the application controls the projected orderings through its index declarations. The primary key defines one projected ordering. Every other index defines the ordering that consists of its columns followed

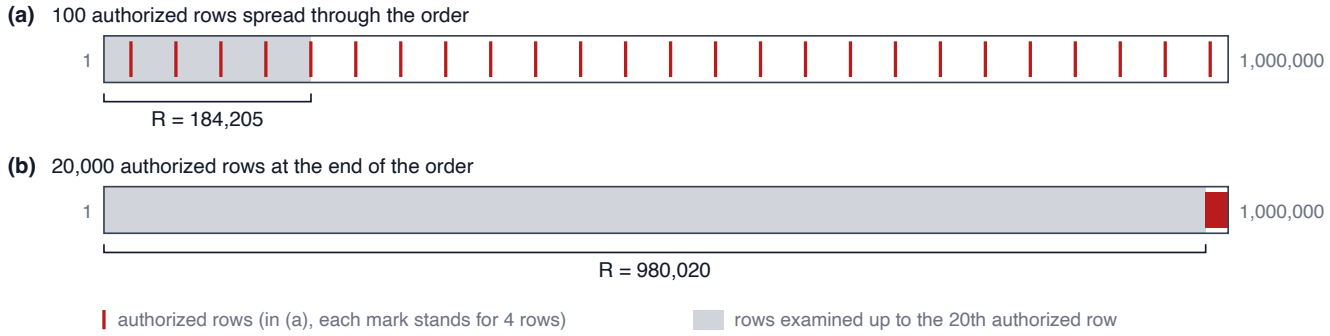
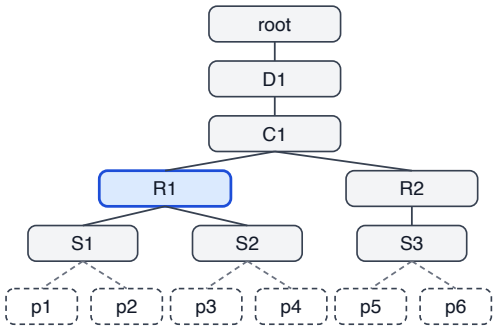


Fig. 1. Rows read by checking each row in price order, for a first page of 20 products in a table of 1,000,000, measured on PostgreSQL 16. (a) A store manager with 100 products spread through the order. (b) A chain manager whose 20,000 products have the highest prices. Each shaded interval is the part of the index read before the 20th authorized product.

(a) Resource tree



(b) Products

Id	Price	ResourceId
p1	3	S1
p2	9	S1
p3	5	S2
p4	7	S2
p5	1	S3
p6	2	S3

(c) Projection on (Price, Id)

Node	Price	Id	
S1	3	p1	
S1	9	p2	
S2	5	p3	
S2	7	p4	
S3	1	p5	
S3	2	p6	
R1	3	p1	range of R1
R1	5	p3	
R1	7	p4	
R1	9	p2	
R2	1	p5	
R2	2	p6	
C1, D1, root: all six			

Fig. 2. The running example. (a) The resource tree; the dashed boxes are rows of the Products table, each naming the store it belongs to. (b) The Products table. (c) Its projection on (Price, Id), which lists each product under each active ancestor of its store, in price order within each resource. The highlighted entries are the range of region R1; C1, D1 and the root hold entries for all six products.

by the primary key; an index that already ends with the primary key defines the same ordering as the index without it, and uniqueness, filters, descending columns and included columns are ignored. A B-tree can be read in either direction, so one projection serves ascending and descending pages. Two kinds of index serve lookups rather than sorting and are not projected: indexes that contain ResourceId, and indexes whose columns are exactly the columns of a foreign key, such as an index on StoreId that finds the products of a store. An index with a column that cannot be part of a projection key, such as a nullable column, is not projected either.

A query uses a projection only if its ordering, with the primary key appended, matches the projection's columns exactly, so sorting by name and then city requires an index on (name, city). A query whose ordering has no projection is served by the first step of the algorithm alone, at a cost of $O(R)$ rows.

C. The Resources That Authorize a Caller

The first piece of information of Section III-D is the set of resources at which the caller holds the requested permission, reduced to the highest ones. Lee holds *view products* at R1 through Viewer and at S2 through Manager, but a grant at S2 adds no products that the grant at R1 does not already give, so only R1 is needed. Dropping such lower grants matters for cost: every resource kept is one range to read.

Definition 4 (Minimal granted scopes). For a caller p, permission $perm$ and time t, a resource x is a qualifying scope if it is active and some $p' \in \mathrm{resolve}(p)$ holds a grant at x that is active at t and whose role carries $perm$. $S(p, perm, t)$ is the set of qualifying scopes x such that no other resource of $\mathrm{anc}^+(x)$ is a qualifying scope. We write $k = |S(p, perm, t)|$.

For Maria and *view products*, $S = \{\mathrm{R1}\}$; for Lee, $S = \{\mathrm{R1}\}$ for *view products* and $S = \{\mathrm{S2}\}$ for *change prices*; for Sam, $S = \{\mathrm{S2}, \mathrm{S3}\}$. The set is computed inside the statement by a table-valued function, the scope function, from the principal identifiers and the permission identifier alone. It applies the same principal, role, validity and activity conditions as the access function and removes every qualifying scope that has another qualifying scope among its active ancestors.

Algorithm 1 Computing a page with ancestor-partitioned index projections.

```
ALGORITHM AuthorizedPage(Q, p, perm, u, P)
  // Q = (T, phi, K); u is the cursor
  pr <- resolve(p); q <- lookup(perm); b <- P
  LOOP
    // Scan
    rows <- first b rows of Q after u in K order,
          each with a(r) = AccessFn(res(r), pr, q)
    A <- those rows with a(r) true
    IF |A| >= P: sel <- first P rows of A; EXIT LOOP
    IF |rows| < b: sel <- A; EXIT LOOP
    // Merge
    IF a projection Pi_K exists:
      FOR EACH x IN ScopeFn(pr, q):
        E[x] <- first b entries of Pi_K under x
                after u whose row satisfies phi
      sel <- first P entries of the union of
             all E[x], in K order
      EXIT LOOP
    b <- 4 * b
  // Verification
  page <- the rows of Q with keys in sel and
          AccessFn(res(r), pr, q), in K order
  cursor <- none if the scan or the merge ran
            out of rows, else K(last entry of sel)
  RETURN (page, cursor)
```

D. Building a Page

Consider Sam's first page for *view products*, sorted by price with $P = 2$. Sam's scopes are S2 and S3. The first two entries under S2 are p3 (5) and p4 (7); the first two under S3 are p5 (1) and p6 (2). Merging the two sorted lists by price gives p5, p6, p3, p4, and the page is p5 and p6. No product outside these two prefixes can belong to the page. Every product Sam may view lies beneath exactly one of Sam's scopes, and within that scope the entries are in price order, so a product that is the third or later entry under its scope has at least two cheaper authorized products ahead of it and cannot be among the two cheapest overall. The next page starts after the cursor (2, p6): under S2 the first two entries after the cursor are still p3 and p4, under S3 there are none, and the page is p3 and p4.

Lee's first page for *view products* reads one range, under R1, and is p1 and p3; the cheaper p5 and p6 are never touched, because they have no entry under R1. Lee's page for *change prices* reads the range under S2 and is p3 and p4. Fig. 3 traces Lee's first page for *view products* through the three steps of the algorithm.

Algorithm 1 gives the complete procedure. It works in rounds with a budget b, the number of rows or entries a step may read in one round, which starts at P. AccessFn is the access function of Section II-D and ScopeFn the scope function of Section IV-C; sel holds the values of K, including the primary key, of the rows selected for the page.

The scan comes first because many callers do not need a projection at all. It reads the next b rows of Q in K order after the cursor and evaluates the access function on each in the same statement. If at least P of them pass, the first P form the page; an administrator with a grant at the root, or a query that already filters to one of the caller's stores, finishes here after reading P rows. If fewer than b rows remain in Q, the query is exhausted and the rows that passed form the last page. For Lee's first page the scan reads p5 and p6, neither passes, and the page is not complete.

The merge finds the page when the scan does not. It runs only when a projection exists for K. For each scope x in S, it reads the first b entries under x that follow the cursor and whose row satisfies φ, and it merges these ranges in K order, keeping the first P entries. Because $b \geq P$, the merge finishes the page in the round in which it runs, by the argument given for Sam above: either it returns P entries, or every range is exhausted and it returns all remaining rows beneath the caller's scopes. The budget grows to $4b$ only for orderings without a projection, for which the scan is repeated with a larger budget.

Verification guarantees that only authorized rows are returned, whatever the projections contain. It loads the selected rows of Q by key, evaluates the access function on each, and returns those that pass, in K order. The cursor is taken from the last row that the scan or the merge selected, so a row removed by verification is not selected again on the next page, and it is empty when the scan reached the end of Q or every range was exhausted. With a projection, a page issues at most five SQL statements: two that resolve the caller's principals and the permission, the scan, the merge and the verification.

E. Keeping Projections Current

A projection is derived from the Products table and the resource tree, so every write that changes either one updates the projection in the same transaction; a committed write is always reflected, and a rolled-back write leaves nothing behind. Inserting or deleting a row, or changing its ResourceId or a projected column, deletes the row's entries through the index on κ and inserts entries for $\mathrm{anc}^{+}(\mathrm{res}(r))$. Moving a resource to a new parent, or deactivating or reactivating it, changes anc^{+} for every row beneath it: when region R1 moves to another chain, every product in R1 has a new chain and a new division above it. The entries of those rows are recomputed by walking the resource's subtree. Fig. 4 shows both kinds of update. Writes that bypass this mechanism, such as bulk loads, require a rebuild that recomputes each projection from T and the resource tables; the same procedure fills the projection of a newly added index.

V. Correctness and Cost

A. Only Authorized Rows Are Returned

Theorem 1 (Soundness). For any contents of the projections, every row returned by Algorithm 1 is authorized for the caller and the permission at the time verification executes.

Proof. Rows are returned only by verification, which evaluates the access function on each selected row. By Theorem 1 of [8], the access function returns a result only for rows that satisfy Definition 1. The scan and the merge determine which rows are selected, and no selected row is returned without passing verification. □

Soundness does not depend on the projections, which affect which rows are found but never which rows are allowed. A projection entry left behind by a failed update, or missing

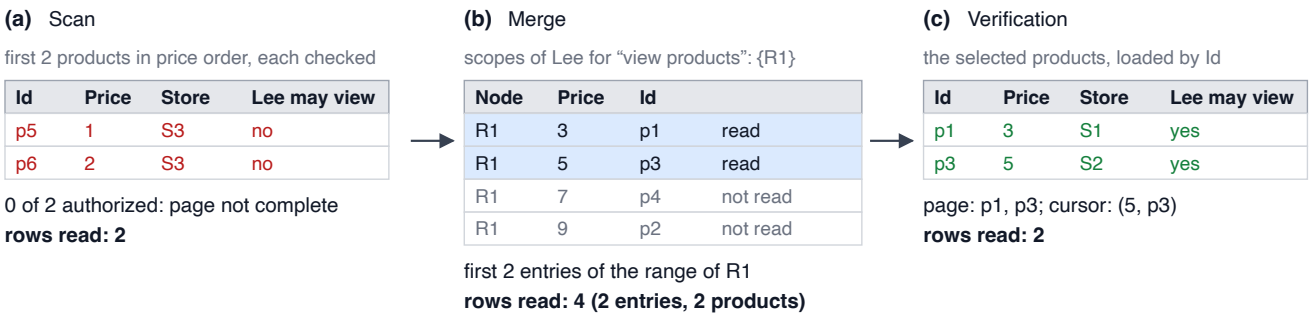


Fig. 3. Lee's first page for view products, sorted by price with a page size of 2, in the example of Fig. 2. (a) The scan reads the two cheapest products, p5 and p6, and the access function rejects both. (b) The scope function returns Lee's single scope R1, and the merge reads the first two entries under R1. (c) Verification loads p1 and p3, checks them, and returns the page with the cursor (5, p3).

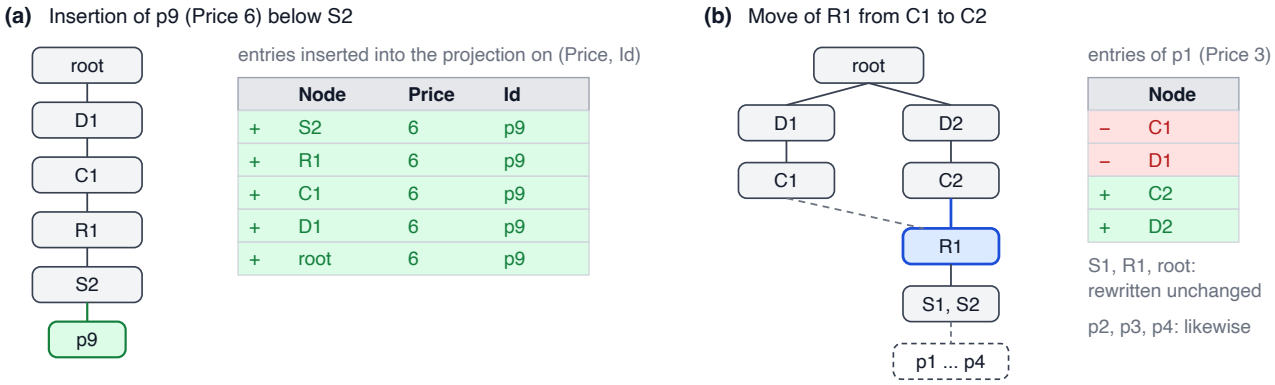


Fig. 4. Maintenance in the example of Fig. 2. (a) Inserting product p9 (price 6) below store S2 adds one entry per active ancestor of S2 to every projection. (b) Moving region R1 from chain C1 to chain C2 rewrites the entries of every row beneath R1; for p1, the entries under C1 and D1 become entries under C2 and D2. Both updates run in the write's transaction.

after a write that bypassed maintenance, can hide a row from a page; it cannot reveal one.

B. Every Authorized Row Is Returned

Completeness depends on the projections being consistent with the data, and on the two lemmas below, which connect the structure of a projection to Definition 1.

Lemma 1 (Projection membership). If Π_K is consistent, then for every row r and resource x, Π_K contains an entry for r under x if and only if $x \in \mathrm{anc}^+(\mathrm{res}(r))$, which is the condition that Definition 1 places on the resource of a grant that authorizes r.

Proof. Immediate from Definition 3 and the definition of consistency. □

Lemma 2 (Disjoint cover). Let $perm$ be unrestricted or restricted to the type of $\mathrm{res}(r)$. Then r is authorized for p and $perm$ at t if and only if some element of $S(p, perm, t)$ lies in $\mathrm{anc}^+(\mathrm{res}(r))$, and at most one element of $S(p, perm, t)$ does.

Proof. If r is authorized, a qualifying scope y lies in $\mathrm{anc}^+(\mathrm{res}(r))$. Let y^* be the qualifying scope on this path that is closest to the root. Since $\mathrm{anc}^+(y^*)$ is the part of the path from y^* upward, it contains no other qualifying scope, so $y^* \in S$. Conversely, an element of S on the path is a qualifying scope, so r is authorized. If two elements of S lay on the same path, the higher one would belong to the active ancestor path of the lower one, which contradicts the minimality of the lower one. □

Lemma 2 is the fact used informally for Sam in Section IV-D: every authorized row lies beneath exactly one of the caller's minimal scopes, so the ranges the merge reads contain all of the caller's rows and none of them twice.

Theorem 2 (Completeness). Suppose Π_K is consistent and $perm$ is unrestricted or restricted to a type shared by the resources of all rows of T. Then Algorithm 1 returns the authorized page of Definition 2, and the cursor it returns continues the enumeration without omitting or repeating an authorized row.

Proof. If the scan finishes the page, it has read the rows of Q after u in K order and evaluated the access function on each, which is sound and complete [8]; its selection consists of the first P authorized rows, or all of them if the query is exhausted. Otherwise the merge runs. By Lemmas 1 and 2, the authorized rows after u that satisfy φ are the disjoint union, over $x \in S$, of the rows with an entry under x after u that

satisfy φ. Each range lists the rows of its scope in K order, so each of the first P rows of the union is among the first P rows of its own range, and reading $b \geq P$ entries from every range and merging them yields the first P rows of the union. If the union contains fewer than P rows, every range is exhausted and all of them are returned. By the same lemmas every selected row is authorized, so verification removes none. The selection contains every authorized row up to its last element, which therefore marks the position from which the next page continues. □

When the assumptions of Theorem 2 fail, the method can return a short page or miss rows, but it still returns only authorized rows. If $perm$ is restricted to a type that some rows beneath a scope do not have, the merge can select such rows, verification removes them, and the page may hold fewer than P rows; the selection still contains every authorized row up to its last element, so the enumeration omits none. If a projection is not consistent, for example after a write that bypassed maintenance, an authorized row whose entry is missing does not appear until the next rebuild, and entries for rows that are no longer authorized are removed by verification. Because the steps are separate statements, a grant or membership revoked between the merge and verification also removes rows from the page, which then reflects the authorization state at verification.

C. The Cost of a Page

Theorem 3 (Page cost). For an ordering with a projection and a query without a filter (φ true for every row), a page costs $O(P \cdot D \cdot M \cdot G_{\max})$ operations in the scan, $O(G \cdot D)$ to compute S, $O(k \cdot P \log k)$ in the merge and $O(P \cdot D \cdot M \cdot G_{\max})$ in verification, a total of $O(P \cdot D \cdot M \cdot G_{\max} + G \cdot D + k \cdot P \log k)$, which is independent of N and σ.

Proof. With a projection, Algorithm 1 completes in its first round, with $b = P$. The scan reads at most P rows through the index on K and evaluates the access function on each. The scope function examines the caller's G qualifying grants and, for each granted resource, at most D active ancestors. The merge performs one seek per scope into the primary key of Π_K, reads at most P entries per scope and the corresponding rows, and merges at most $k \cdot P$ entries with a heap of size k. Verification reads at most P rows by key and evaluates the access function on each. None of these terms depends on N or σ. □

The bound says what a page costs in terms of the caller rather than the table. For Lee, $G = 2$ and $k = 1$, so the merge reads one range of at most P entries; for a caller with grants on 100 stores, $k = 100$ and the merge reads up to $100P$ entries. The scan runs first because it costs at most P rows and settles the page for callers who may see most rows without computing S or touching a projection. The theorem assumes no filter; with a filter that only a fraction f of a scope's rows satisfy, the merge reads about P/f entries of that range, and at most all of its rows, so the cost of a filtered page depends on how selective the filter is within the caller's scopes.

D. Storage and Write Cost

Proposition 1. For a table of N rows with I projected orderings, the projections contain at most $N \cdot (D+1) \cdot I$ entries. Inserting or deleting a row, or changing its ResourceId or a projected column, writes at most $(D+1) \cdot I$ entries. Moving a resource, or changing whether it is active, rewrites the entries of the N_x rows beneath it, at most $N_x \cdot (D+1) \cdot I$ entries. A rebuild writes every entry once.

Proof. Each row has $L(r) \leq D+1$ entries in each projection, and a write changes only the entries of the rows whose anc^+ or projected values it changes. □

E. Cases Outside the Bound

Theorem 3 bounds the page by the caller's grants and scopes, not by a constant, and three kinds of request fall outside the favourable case. A caller with 10,000 grants on individual rows has $G = k = 10{,}000$, and the merge performs 10,000 seeks to find 20 rows (Section VII-E); grants on containers avoid this. A filter that few of the caller's rows satisfy increases the entries read per scope, as in Section V-C. An ordering without a projection is computed by the scan alone, at a cost of $O(R)$ rows.

VI. Implementation

SqlOS [18], an open-source library that provides SHRBAC to applications built on Entity Framework (EF) Core, the object-relational mapper of .NET, implements the method for SQL Server and PostgreSQL. Both engines use the same tables, statement shapes and tests, with engine-specific SQL where the dialects differ.

A. What an Application Declares

An application declares indexes as it normally would and requests pages through one method; the library creates and maintains the projections.

```
modelBuilder.Entity<Product>(e =>
{
    e.HasIndex(p => p.ResourceId);
    e.HasIndex(p => new { p.Price, p.Id });
    e.HasIndex(p => new { p.Status, p.CreatedAt });
});

var page = await db.Products
    .Where(p => p.Name.Contains(term))
    .OrderBy(p => p.Price)
    .ToPageAsync(principalId, "product.view",
                 pageSize: 20, cursor);
```

ToPageAsync returns the page together with a cursor, and the application passes that cursor back to request the following page. The library adds each projection to the application's EF Core model as a table in its own schema, so the application's migrations or schema tools create it alongside the application's tables. It derives the projected orderings from the index declarations of every entity type that carries a ResourceId, by the rules of Section IV-B. Projection columns copy the type, length, precision, collation and value conversion of their source columns, so a projection sorts values exactly as the source index does. An index or entity type that cannot be projected, for example because a column is nullable or the

entity is mapped to a view, is reported once at startup. A query whose ordering has no projection is served by the scan, and the library logs a warning that names the index to declare.

B. The SQL Statements

Each step of Algorithm 1 is one SQL statement, and no set derived from the caller's grants leaves the database. The scan evaluates the access function in the select list of the application's query, as an EXISTS expression, and limits the query to b rows. The merge joins the scope function, `fn_PageNodes`, to a correlated subquery, which the database evaluates once per scope and which reads the first b projection entries after the cursor whose row satisfies φ. On SQL Server the subquery is CROSS APPLY with TOP and on PostgreSQL JOIN LATERAL with LIMIT, so each scope costs one bounded range scan of the projection's primary key. EF Core translates the simpler formulation of a per-scope limit into a join with a window function that numbers every entry of the projection, which reads the whole projection before any limit applies, so the library builds the correlated subquery itself. Verification passes the selected keys as one parameter, through OPENJSON on SQL Server and an array on PostgreSQL. Two statements resolve the caller's principals and the permission at the start of every page, so each page sees the grants and group memberships committed before it. The ancestor walk returns no rows for a resource whose chain exceeds the configured depth or contains a cycle, so the rows of such a resource are neither authorized nor projected.

The scope function is written differently on the two engines because one formulation was slow on PostgreSQL. SQL Server uses a correlated NOT EXISTS to drop scopes that have a qualifying ancestor. On PostgreSQL that formulation rescanned the materialized set of granted resources once for each granted resource and took 6.5 s for a caller with 10,000 grants, whereas a set difference (EXCEPT) took 0.3 s. The PostgreSQL function also declares a fixed row estimate and a SET clause that disables just-in-time compilation, the step in which PostgreSQL compiles an expensive query to machine code before running it, which costs more than it saves on short queries. A function with a SET clause is not inlined into the calling query; when it was inlined, the estimated cost of page queries exceeded PostgreSQL's compilation threshold, and compilation added about 500 ms to pages whose execution took about 20 ms.

C. Maintenance in the Save

Projections are maintained inside the application's save, so a write and its projection entries commit or roll back together. Before a save, the library's database context checks whether the pending changes include a resource or a row of a protected table, and if they do not, the save proceeds unchanged. Otherwise the context opens a transaction or, if the application already has one, a savepoint within it, so that the library can undo its own work without ending the application's transaction. It then performs the save, collects the rows and resources that the save inserted, deleted or modified, and issues one command that updates every affected projection. Collecting changes after the save captures values set by save interceptors, cascade deletions and keys generated by the database. A failure in either step rolls back both.

Two implementation issues emerged during testing with one million rows on PostgreSQL. The estimated cost of the subtree statements, a recursive walk combined with the inlined ancestor function, exceeded the threshold for just-in-time compilation, and inserting a single row took 165 ms instead of 1.6 ms. The maintenance command therefore disables just-in-time compilation for its own statements and restores the transaction's setting afterwards. Walking subtrees from every changed resource, including resources added in the same save, led the optimizer to estimate a save of 1,000 new resources at millions of rows and to hash the entire projection. Because a resource's parent is a foreign key, a resource added or deleted in a save has no descendants outside that save, so the implementation walks only resources whose parent or activity changed. The maintenance command for that save fell from 2.6–4.9 s to 66–111 ms.

Writes that bypass the save are repaired by a rebuild, which deletes and rewrites each projection in its own transaction, under an application lock and without a command timeout. At startup, the library fills every projection that is empty while its source table has rows, which is the state after an index is added.

VII. Evaluation

A. Questions

The analysis of Section III predicts that checking rows in order costs R rows per page and collecting through a closure costs about V, and Theorem 3 predicts that projections cost about P entries per scope. The experiments test these predictions on one table of 1,000,000 products and ask five questions.

- **RQ1.** How does the cost of a page depend on how many rows the caller may see and on where they fall in the sort order, for checking rows in order and for projections?
- **RQ2.** What do projections cost for callers who may see most rows, and for queries that already filter to one of the caller's scopes?
- **RQ3.** How does the cost of projections grow with the number of scopes a caller holds?
- **RQ4.** How do projections compare with collecting the caller's rows through an unordered ancestor closure?
- **RQ5.** What do projections cost in storage and in write latency?

B. Setup

Workload. The data is a retail hierarchy like the example, at scale: a root, 5 divisions, 50 chains, 500 regions, 10,000 stores and 1,000,000 products, with 10 chains per division, 10 regions per chain, 20 stores per region and 100 products per store. Each product is a resource of its own below its store, so every product has six active ancestors including itself. Prices are uniform over the products, except that the 20,000 products

of one chain carry the highest prices, which places that chain's products at the end of the price order. The Products table has indexes on (Price, Id) and (StoreId, Price, Id) and a unique index on ResourceId. Every query orders by (Price, Id), uses a page size of 20 and requests the first page.

Callers. Seven callers (Table III) cover the cases of Section III, from an administrator who may see every product to a chain manager whose products all sort last. Each holds a role that carries the permission *product.view* at the resources named in the table. The scoped store manager is the store manager with a query that also filters on the store's StoreId, as a list of one store's products would. For RQ3, five further callers hold grants on 1, 10, 45, 100 and 500 stores whose products are spread through the price order.

Methods. Three methods compute each page.

- **A, checking rows in order** [8], the method of Section III-B: the application's query with EXISTS over the access function, ordered by (Price, Id) and limited to 20 rows, executed as one statement. As in [8], the caller's principals are resolved before timing starts.
- **B, projections**: Algorithm 1, including the two statements that resolve the caller's principals and the permission.
- **C, closure enumeration**, the combination of Section III-C: in rounds with budgets $b = 20, 80, 320, \ldots$, C checks the next b rows in order as A does; once the caller's minimal scopes hold at most b entries in a key-only projection, which is the ancestor closure of Section III-C, it instead reads those rows by key, sorts them by (Price, Id) and returns the first 20 after the cursor. The page comes from whichever path finishes first, the row keys pass from the closure statement to the fetch statement as a parameter, and C reads $O(\min(R, V) + P)$ rows.

Measurements. Every page returns 20 products (or fewer if fewer qualify), so returned rows do not distinguish the methods. The measure that does is *rows read*: the number of rows produced by the execution plan's operators over the Products table and the projection tables, summed over all statements of a page, including rows that a filter or the access function later rejected. It is taken from the actual plans (SET STATISTICS XML on SQL Server; EXPLAIN (ANALYZE, FORMAT JSON) on PostgreSQL) and is exact and independent of the speed of the machine. On SQL Server the plan of A reads each examined row twice, once from the price index and once through a key lookup for ResourceId, so its counts are double those of PostgreSQL. *Page latency* is the time to return the page, measured as the median of up to 15 runs within a 60-second budget after a warm-up run; a run longer than one minute was measured once.

Platform. All measurements ran on one Apple Silicon laptop with Docker Desktop. PostgreSQL 16 ran natively with its default settings, including just-in-time compilation. SQL Server 2022 ran as the x64 image under CPU emulation, which adds tens of milliseconds to each statement; its row counts are exact, but its latencies are not representative of native execution and are compared only within the engine.

Artifacts. The implementation, benchmark harness and data generator are part of SqlOS [18]. The comparison of A and B on both engines (Tables IV and V) was measured with version 10.0.0 [19], in which the price projection was declared explicitly and no other ordering was projected; version 11.0.0 derives the same projection from the price index and issues the same page statements. A repetition on PostgreSQL with version 11.0.0 reproduced every row count of Table V, with B latencies of 14.0, 18.3, 18.6, 24.9, 457.5, 16.4 and 12.4 ms in the order of Table III [20]. Method C and the sweep over granted stores (Tables VI and VII) were measured with an experimental build of the same harness on the same machine and data [21], with A and C in one run and B in a second; in these runs A and B read the same numbers of rows as in Tables IV and V. The row counts of C differed between runs because its plans depend on optimizer statistics; the region manager read 13,263 rows in one run and 12,544 in another. Storage and write costs (Tables VIII and IX) were measured with version 11.0.0 [20].

C. RQ1: Selectivity and Position

Tables IV and V give A and B for the seven callers on the two engines, and Fig. 5 plots the rows read by all three methods on PostgreSQL. Under A, the rows read for a page equal R of Table III and range from 20 to 980,020 depending on where the caller's products fall. Under B, no caller with a single scope reads more than 80 rows on PostgreSQL or 100 on SQL Server. For the store manager, the rows read fall from 184,205 to 80 on PostgreSQL and from 368,410 to 100 on SQL Server, and the latency from 8,420 ms to 22.6 ms and from 179,558 ms to 89.0 ms. The region manager reads 80 rows instead of 9,306. The far-end caller, whose products all sort last, reads 980,020 rows under A and 80 under B on PostgreSQL, and 1,960,040 and 100 on SQL Server; its latency falls from 44,208 ms to 16.4 ms and from 942,593 ms to 90.3 ms.

The 80 rows of a single-scope caller under B are the 20 the scan reads and rejects, 40 in the merge (20 projection entries and the 20 products on which the filter is evaluated) and 20 in verification; SQL Server reads 100 because its scan reads each row twice. Position has no effect under B because the range of the far-end caller's chain begins with its cheapest product, exactly as the store manager's range does.

D. RQ2: Callers Who Need No Projection

For callers who may see most rows, B costs 7 to 9 ms more than A on PostgreSQL and 21 to 29 ms more on SQL Server. The administrator's page takes 14.0 ms instead of 5.6 ms on PostgreSQL and 52.3 ms instead of 23.7 ms on SQL Server, and reads 40 rows instead of 20 on PostgreSQL. The scoped store manager's page takes 12.8 ms instead of 5.4 ms and 39.4 ms instead of 18.4 ms. In both cases the scan finishes the page and B reads no projection. The difference is the number of statements: A executes one, and B executes four, of which

TABLE III

CALLERS OF THE EVALUATION. "MAY SEE" IS THE NUMBER OF PRODUCTS THE CALLER IS AUTHORIZED TO VIEW, V. "ROWS TO 20TH HIT" IS R, THE NUMBER OF PRODUCTS IN PRICE ORDER UP TO AND INCLUDING THE 20TH ONE THE CALLER MAY SEE, MEASURED ON POSTGRESQL.

Caller	Grants	May see (V)	Share of table	Rows to 20th hit (R)
Administrator	one, on the root	1,000,000	100%	20
Region manager	one, on a region	2,000	0.2%	9,306
Store manager	one, on a store	100	0.01%	184,205
100 stores	one on each of 100 stores	10,000	1%	2,233
10,000 single products	one on each of 10,000 products	10,000	1%	1,452
Far end	one, on the chain with the highest prices	20,000	2%	980,020
Scoped store manager	one, on a store; query filtered by StoreId	100	0.01%	20

TABLE IV

SQL SERVER 2022, FIRST PAGE OF 20 OF 1,000,000 PRODUCTS SORTED BY PRICE. ROWS READ ARE EXECUTION-PLAN ROW COUNTS (SECTION VII-B).

Caller	A: page latency (ms)	A: rows read	B: page latency (ms)	B: statements	B: rows read
Administrator	23.7	40	52.3	4	60
Region manager	9,032	18,612	88.2	5	100
Store manager	179,558	368,410	89.0	5	100
100 stores	2,227	4,466	93.7	5	4,060
10,000 single products	1,420	2,904	762.7	5	20,060
Far end	942,593	1,960,040	90.3	5	100
Scoped store manager	18.4	40	39.4	4	60

TABLE V

POSTGRESQL 16, FIRST PAGE OF 20 OF 1,000,000 PRODUCTS SORTED BY PRICE. ROWS READ ARE EXECUTION-PLAN ROW COUNTS (SECTION VII-B).

Caller	A: page latency (ms)	A: rows read	B: page latency (ms)	B: statements	B: rows read
Administrator	5.6	20	14.0	4	40
Region manager	416.8	9,306	18.6	5	80
Store manager	8,420	184,205	22.6	5	80
100 stores	107.6	2,233	21.8	5	4,040
10,000 single products	70.8	1,452	390.2	5	20,040
Far end	44,208	980,020	16.4	5	80
Scoped store manager	5.4	20	12.8	4	40

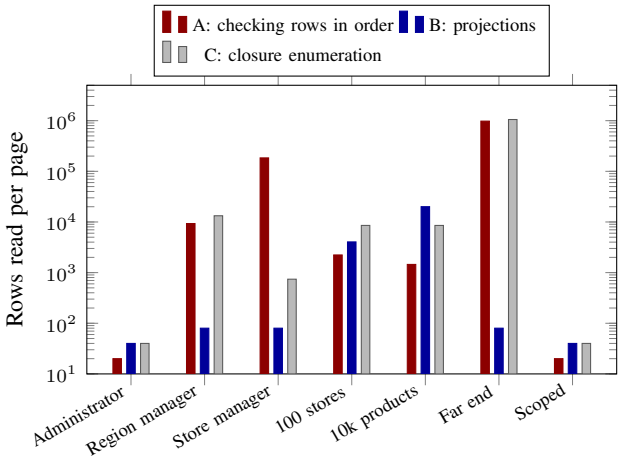


Fig. 5. Rows read for a first page of 20 on PostgreSQL 16, by checking rows in order (A), projections (B) and closure enumeration (C); logarithmic scale. Values are the execution-plan row counts of Tables V and VI.

two resolve the caller's principals and the permission, which A performs before timing starts, and the other two scan 20 rows and verify them.

E. RQ3: Number of Scopes

The cost of B grows with the number of scopes, at about 40 rows read per scope on PostgreSQL. The caller with grants on 100 stores reads 4,040 rows under B, 40 per scope plus 40 in the scan and verification, in 21.8 ms against 107.6 ms under A. The caller with 10,000 grants on single products reads 20,040 rows under B, because each of its 10,000 scopes holds one product and the merge reads each one. A reads 1,452 rows for this caller, whose products are spread evenly through the price order, in 70.8 ms against 390.2 ms for B. On SQL Server the latencies are reversed, 1,420 ms for A and 762.7 ms for B, although B reads 20,060 rows against 2,904.

The sweep over 1 to 500 granted stores (Table VII and Fig. 6) shows where the two methods cross. The rows read by A fall from 193,501 to 367 as the caller's share of the table grows, roughly in proportion to P/σ. The rows read by B rise as $40k + 40$ for k granted stores, from 80 to 20,040, and its latency from 15.5 ms to 38.9 ms. B reads more rows than A from 100 stores on but is slower only at 500 stores, where A takes 24.0 ms and B 38.9 ms. A row read costs more under A, which applies the access function to every row it reads, than in B's merge, which reads projection entries and products without it. A caller with one grant on a higher resource, such as the region manager or the far-end caller, reads 80 rows under B

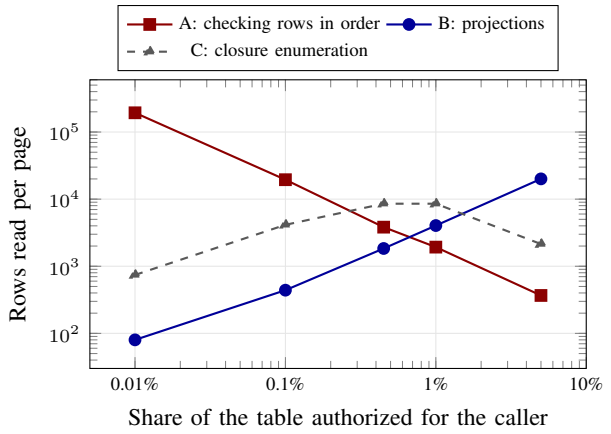


Fig. 6. Rows read for a first page of 20 on PostgreSQL 16 against the share of the table the caller may see, for callers with grants on 1 to 500 stores; logarithmic scales. Values are from Table VII.

whatever its share of the table, because the number of scopes, not the number of products, sets the cost.

F. RQ4: Projections Against Closure Enumeration

Table VI reports C for the seven callers. Closure enumeration removes the effect of position, but it reads all of the caller's rows and its cost depends on the plan the optimizer picks. On PostgreSQL, C reads 2.0 to 7.4 times $\min(R, V)$ rows for six of the seven callers, in 4 to 16 statements per page. For the far-end caller, C's fetch statement requested the caller's 20,000 rows by key. PostgreSQL's cost estimates for a parallel sequential scan of Products and for 20,000 primary-key lookups differed by 7%, the optimizer chose the scan, and the page read 1,054,147 rows; with parallel query disabled, the same page read 74,145 rows in 2,603.1 ms. On SQL Server, C read 101,445 rows for this caller. For the caller with 10,000 single grants, C recomputes the minimal scopes in each round; on SQL Server, the row limit of the closure statement propagated into the minimality computation, and one round took 25 s of the 35,526.3 ms.

Where C reads 742 to 1,054,147 rows for the region manager, the store manager and the far-end caller, B reads 80 for each. C must read all of the caller's rows before it can sort them, whereas B reads the first 20 entries of a range that is already sorted. For the caller with 10,000 single grants, C reads fewer rows than B, 8,544 against 20,040, but takes 2,305.1 ms against 390.2 ms. In the sweep (Table VII), C reads fewer rows than A for the callers with 1 and 10 stores and more for those with 45, 100 and 500, and more than B for every caller except the one with 500 stores. Sorting each scope's entries by the index key, as projections do, removes the dependence on both the position and the number of the caller's rows and leaves a dependence on the number of scopes.

G. RQ5: Storage and Write Cost

Each projection of the 1,000,000-product table holds 6,000,000 entries, one per product and active ancestor, and occupies 550 MB to 866 MB (Table VIII). Version 11.0.0 projects the primary key and both indexes, so the table has three projections. Writing all 18,000,000 entries with the rebuild took 175 s on PostgreSQL; with version 10.0.0, building the price projection alone took 55 s on PostgreSQL and 200 s on SQL Server.

Table IX gives the write cost. Inserting one product with its resource writes six entries into each of the three projections and takes 5.7 ms against 1.4 ms for a table without projections. Inserting 1,000 products in one save takes 635.6 ms against 211.8 ms. Moving a region of 2,000 products to another chain rewrites 12,000 entries in each projection and takes 501.0 ms against 5.2 ms. Both columns were measured with the two maintenance changes of Section VI-C in place.

H. Threats to Validity

The main threats are the single machine, the emulated SQL Server and the synthetic data. All measurements ran on one laptop, and latencies are medians of a small number of runs on a shared machine; the conclusions rest on the row counts, which are exact. The data has uniform prices, equal fan-out at every level and one resource per product, which gives every product the longest ancestor path of the tree. Skew in the price distribution changes R but not the kP term of B. All measurements request the first page; deep cursors were not measured, although the analysis of Section V does not depend on the cursor. On PostgreSQL, statements were planned at every execution because the client did not prepare them. The measurements combine runs of two builds whose statements for A and B are identical, as the identical row counts confirm, and method C is not part of a released version. Integration tests on both engines compare every page of B with row-by-row point checks [18].

VIII. Related Work

A. Authorization Services

Authorization services are built for single checks, and those that list objects return unsorted sets. Zanzibar [5] serves point checks at large scale and describes search as checking candidates, noting that search results "often require tens to hundreds of checks". Clients that need other access paths maintain secondary indexes from its change stream, and its Leopard index precomputes flattened group membership. Services in the Zanzibar family list objects by expanding the relationship graph backwards from the user. OpenFGA's ListObjects returns an unordered set bounded by a result limit and a deadline [9], and its maintainers state that it cannot be paginated because "the graph evaluation is done concurrently (and thus is not ordered)" [22]. Auth0's documentation combines database queries, batch checks and list calls, and notes that a partial list "is not enough, because you won't be able to sort using it" [23].

Services that support sorting copy permissions into the application's database. AuthZed Materialize streams computed permissions into that database so that applications can "sort, filter, and paginate over many thousands of authorized objects natively" [24], and Auth0 FGA documents a precomputed

TABLE VI
CLOSURE ENUMERATION (C) ON BOTH ENGINES, FIRST PAGE OF 20 OF 1,000,000 PRODUCTS SORTED BY PRICE. $\min(R, V)$ IS THE BOUND OF SECTION III-C. THE LAST COLUMN REPEATS THE ROWS READ BY B FROM TABLES IV AND V.

Caller	$\min(R, V)$	C PostgreSQL: page latency (ms)	C PostgreSQL: rows read	C SQL Server: page latency (ms)	C SQL Server: rows read	B: rows read (PostgreSQL / SQL Server)
Administrator	20	13.7	40	50.9	60	40 / 60
Region manager	2,000	951.3	13,263	2,644.3	19,364	80 / 100
Store manager	100	59.3	742	245.1	1,162	80 / 100
100 stores	2,233	875.2	8,549	2,701.8	15,364	4,040 / 4,060
10,000 single products	1,452	2,305.1	8,544	35,526.3	15,364	20,040 / 20,060
Far end	20,000	3,044.9	1,054,147	12,116.5	101,445	80 / 100
Scoped store manager	20	13.8	40	53.2	60	40 / 60

TABLE VII
SWEEP OVER THE NUMBER OF GRANTED STORES ON POSTGRESQL 16, FIRST PAGE OF 20 SORTED BY PRICE. A AND C WERE MEASURED IN ONE RUN AND B IN A SECOND (SECTION VII-B).

Granted stores	May see (V)	Share of table	$\min(R, V)$	A: rows read	A: page latency (ms)	B: rows read	B: page latency (ms)	C: rows read	C: page latency (ms)
1	100	0.01%	100	193,501	11,707.8	80	15.5	742	59.3
10	1,000	0.1%	1,000	19,481	1,295.4	440	16.3	4,142	168.6
45	4,500	0.45%	3,825	3,825	204.3	1,840	17.8	8,549	883.3
100	10,000	1%	1,928	1,928	105.5	4,040	20.2	8,549	899.6
500	50,000	5%	367	367	24.0	20,040	38.9	2,142	200.0

TABLE VIII
PROJECTIONS OF THE PRODUCTS TABLE ON POSTGRESQL 16, 1,000,000 PRODUCTS WITH SIX ACTIVE ANCESTORS EACH.

Ordering	Entries	Size (MB)
(Price, Id)	6,000,000	706
(StoreId, Price, Id)	6,000,000	866
(Id)	6,000,000	550

TABLE IX
WRITE LATENCY ON POSTGRESQL 16 FOR A PRODUCTS TABLE WITHOUT PROJECTIONS AND FOR THE SAME TABLE WITH THREE PROJECTIONS (MEDIAN; 25 SAVES FOR THE SINGLE INSERTION).

Operation	Without projections (ms)	With three projections (ms)
Insert one product	1.4	5.7
Insert 1,000 products in one save	211.8	635.6
Move a region of 2,000 products	5.2	501.0

permissions index [23]. These copies hold permission facts, which rows a user may see, and are updated asynchronously from a change stream. Projections hold structure, which rows lie beneath which resource, so they change with rows and resources but not with grants, and they are updated in the writing transaction. Partitioning data by the scopes a user may read, as projections do, also appears in search systems: Dropbox's search engine determines at query time the namespaces a user may read and retrieves documents within them [25], and Slack keeps a separate indexed copy of a message for each organization that shares a channel [26].

B. Row-Level Security and Query Filters

Row-level security and policy compilers filter rows with predicates, and when the authorized set is not a condition on an indexed column, as with grants at arbitrary levels of a hierarchy, their cost follows the rows a query reads, as in Section III-B. Query modification adds authorization predicates to queries [27], and row-level security in PostgreSQL and SQL Server builds it into the engine. PostgreSQL evaluates a policy expression for each row "prior to any conditions or functions coming from the user's query" [28]. Supabase's guidance states that the cost scales with the rows a query scans and recommends indexing the columns that policies filter on and evaluating functions once per statement [29]. Microsoft's guidance for SQL Server row-level security is to avoid recursion in predicate functions [30], and practitioners recommend flattening hierarchies before using them in predicates [31]. Policy engines compile policies into query conditions: Oso generates SQL filters [32], Cerbos returns a condition tree for translation into a query [33], and Open Policy Agent derives filters by partial evaluation [34].

C. Hierarchy Encodings and Merging

Hierarchy encodings make every subtree contiguous in one order, the order of the hierarchy itself, whereas a projection makes it contiguous in the order of an index. Closure tables store one row per ancestor–descendant pair, which makes subtree queries simple and makes moves rewrite the pairs of the moved subtree [35]. The hierarchyid type of SQL Server orders nodes depth-first, so a depth-first index places every subtree contiguously [36]; Spanner's interleaved tables store child rows under their parent's key [37]; and nested intervals encode containment in numeric intervals [38]. A projection is a closure table extended with the key of an index and kept once for every projected ordering. The merge step of Algorithm 1 is a k-way merge of sorted ranges. The threshold algorithm

of Fagin et al. [39] addresses the more general problem of combining scores from several sorted lists; Algorithm 1 needs only a merge because each authorized row lies in exactly one range.

IX. Limitations and Future Work

The method trades storage and write time for read time. With three projected orderings and six active ancestors, each product has eighteen projection entries, which every insertion writes. Indexes created for filtering, such as (StoreId, Price, Id), are projected as well; that projection occupies 866 MB in Table VIII although no measured query sorts by it, and the implementation has no way to exclude an index. Moving a resource rewrites the entries of every row beneath it.

Some orderings and grant patterns fall outside the bound. Sort columns must be non-nullable, because they form part of the projection's primary key, and an ordering on a nullable column is served by the scan alone. A caller with many separate grants pays one range read per scope (Section V-E). A permission restricted to a resource type that some rows beneath a caller's scopes do not have can produce pages with fewer than P rows (Section V-B). In a hierarchy of entity types mapped to one table, only the indexes of the root type are projected.

Projections are only as current as the writes that maintain them. Writes that bypass the library's database context, such as bulk loads and raw SQL statements, leave projections inconsistent until a rebuild. Under snapshot isolation, a save that moves or deactivates a resource and a concurrent save that changes rows beneath it can leave those rows' entries on the old path, and under locking isolation either save can fail with a deadlock or a duplicate key. Rows with stale entries can be missing from pages until a rebuild, and verification prevents stale entries from exposing rows; the implementation does not take a lock that every save would contend on.

Several extensions follow from these limitations. Maintaining projections inside the database engine, through statement-level triggers or an extension, would keep them consistent for every writer; Definition 3 and the rebuild procedure specify the required result. A SQL function that returns an authorized page would make the method available to clients that do not use the library. Encoding nulls as ordered key values would allow nullable sort columns. Omitting the row lookups of the merge when Q has no filter would halve the rows that step reads. Measurements on native SQL Server hardware, larger tables, skewed distributions and deep cursors would complement Section VII.

X. Conclusion

Under scoped role-based access control, the rows a person may see are the rows beneath a small set of resources determined by the person's grants and the requested permission, while the order in which a list shows them comes from an index that sorts the whole table. Checking rows in sort order reads every row up to the P-th one the person may see, and collecting the person's rows through an ancestor closure reads all of them. Ancestor-partitioned index projections keep the rows beneath every resource sorted by every index, so a page reads at most P entries per authorizing resource, in $O(P \cdot D \cdot M \cdot G_{\max} + G \cdot D + k \cdot P \log k)$ operations, and returns only rows that pass the original access check. With 1,000,000 products on two engines, callers with one scope read at most 100 rows per page wherever their rows fall in the order. The price is one entry per row, ancestor and projected ordering, written with every change, and one range read per scope for callers with many separate grants.

AI Disclosure

In accordance with IEEE policy, the author discloses that AI tools (Claude, Anthropic) were used to assist with manuscript preparation and editing and with the implementation and benchmarking work described. The technical decisions and the experimental design are the work of the author.

References

[1] Microsoft, "What is Azure role-based access control (Azure RBAC)?," Microsoft Learn. https://learn.microsoft.com/en-us/azure/role-based-access-control/overview (accessed Oct. 2026).

[2] Microsoft, "Understand scope for Azure RBAC," Microsoft Learn. https://learn.microsoft.com/en-us/azure/role-based-access-control/scope-overview (accessed Oct. 2026).

[3] WorkOS, "Fine-Grained Authorization (FGA)," WorkOS Documentation. https://workos.com/docs/fga/index (accessed Oct. 2026).

[4] OpenFGA, "Parent-Child Objects," OpenFGA Documentation. https://openfga.dev/docs/modeling/parent-child (accessed Oct. 2026).

[5] R. Pang, R. Caceres, M. Burrows, Z. Chen, P. Dave, N. Germer, A. Golynski, K. Graney, N. Kang, L. Kissner, J. L. Korn, A. Parmar, C. D. Richards, and M. Wang, "Zanzibar: Google's Consistent, Global Authorization System," in *Proc. USENIX ATC*, pp. 33–46, 2019.

[6] R. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[7] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST Standard for Role-Based Access Control," *ACM Trans. Information and System Security*, vol. 4, no. 3, pp. 224–274, 2001.

[8] R. Slaney, "Database-Native Authorization for Human and Autonomous Principals: The SHRBAC Model," in *Proc. IEEE COMPSAC*, 2026.

[9] OpenFGA, "Configuration," OpenFGA Documentation (listObjectsDeadline, listObjectsMaxResults). https://openfga.dev/docs/getting-started/setup-openfga/configuration (accessed Oct. 2026).

[10] AuthZed, "Protecting a list endpoint," SpiceDB Documentation. https://authzed.com/docs/spicedb/modeling/protecting-a-list-endpoint (accessed Oct. 2026).

[11] Microsoft, "Understand Azure role assignments," Microsoft Learn. https://learn.microsoft.com/en-us/azure/role-based-access-control/role-assignments (accessed Oct. 2026).

[12] Microsoft, "Understand Azure role definitions," Microsoft Learn. https://learn.microsoft.com/en-us/azure/role-based-access-control/role-definitions (accessed Oct. 2026).

[13] Microsoft, "Organize your resources with management groups," Microsoft Learn. https://learn.microsoft.com/en-us/azure/governance/management-groups/overview (accessed Oct. 2026).

[14] Microsoft, "List Azure deny assignments," Microsoft Learn. https://learn.microsoft.com/en-us/azure/role-based-access-control/deny-assignments (accessed Oct. 2026).

[15] Microsoft, "What is Privileged Identity Management?," Microsoft Learn. https://learn.microsoft.com/en-us/entra/id-governance/privileged-identity-management/pim-configure (accessed Oct. 2026).

[16] Z. Zhang, X. Zhang, and R. Sandhu, "ROBAC: Scalable Role and Organization Based Access Control Models," in *Proc. CollaborateCom*, 2006.

[17] N. Solanki et al., "Resource and Role Hierarchy Based Access Control for Resourceful Systems," in *Proc. IEEE COMPSAC*, pp. 396–401, 2018.

[18] R. Slaney, "SqlOS," open-source library (MIT license), version 11.0.0, Oct. 2026. https://github.com/ross-slaney/sqlos

[19] R. Slaney, benchmark results for SqlOS 10.0.0 on SQL Server 2022 and PostgreSQL 16, ross-slaney/sqlos pull request 516, Oct. 2026. https://github.com/ross-slaney/sqlos/pull/516

[20] R. Slaney, benchmark results for SqlOS 11.0.0 on PostgreSQL 16, ross-slaney/sqlos pull request 524, Oct. 2026. https://github.com/ross-slaney/sqlos/pull/524

[21] R. Slaney, benchmark results for closure enumeration and the selectivity sweep, ross-slaney/sqlos pull request 522, Oct. 2026. https://github.com/ross-slaney/sqlos/pull/522

[22] OpenFGA maintainers, reply on ListObjects pagination, OpenFGA GitHub discussion 105, Jan. 2023. https://github.com/orgs/openfga/discussions/105

[23] Auth0, "Search with permissions," Auth0 FGA Documentation. https://docs.fga.dev/integration/advanced/search-with-permissions (accessed Oct. 2026).

[24] AuthZed, "AuthZed Materialize," AuthZed Documentation. https://authzed.com/docs/authzed/concepts/authzed-materialize (accessed Oct. 2026).

[25] Dropbox, "Architecture of Nautilus, the new Dropbox search engine," Dropbox Tech Blog, Sept. 2018. https://dropbox.tech/machine-learning/architecture-of-nautilus-the-new-dropbox-search-engine

[26] Slack, "Slack search powers visibility and compliance across your organisation," Slack Blog. https://slack.com/blog/developers/enterprise-ai-search-with-slack-search

[27] M. Stonebraker and E. Wong, "Access Control in a Relational Data Base Management System by Query Modification," in *Proc. ACM Annual Conf.*, pp. 180–187, 1974.

[28] PostgreSQL Global Development Group, "Row Security Policies," PostgreSQL Documentation. https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[29] Supabase, "Row Level Security performance," Supabase Documentation. https://supabase.com/docs/guides/database/postgres/row-level-security-performance

[30] Microsoft, "Row-Level Security," SQL Server Documentation. https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security

[31] B. Johnston, "SQL Server RLS performance and troubleshooting," Redgate Simple Talk, Sept. 2023. https://www.red-gate.com/simple-talk/databases/sql-server/sql-server-rls-performance-and-troubleshooting/

[32] Oso, "Local Authorization," Oso Documentation. https://www.osohq.com/docs/develop/facts/local-authorization

[33] Cerbos, "Filtering resources by permission," Cerbos Documentation. https://docs.cerbos.dev/cerbos/latest/recipes/filtering-resources

[34] Open Policy Agent, "Data Filtering," OPA Documentation. https://www.openpolicyagent.org/docs/filtering/

[35] B. Karwin, "Moving Subtrees in Closure Table Hierarchies," Percona Blog, Feb. 2011. https://www.percona.com/blog/moving-subtrees-in-closure-table/

[36] Microsoft, "Hierarchical Data (SQL Server)," SQL Server Documentation. https://learn.microsoft.com/en-us/sql/relational-databases/hierarchical-data-sql-server

[37] Google Cloud, "Schemas overview," Spanner Documentation. https://docs.cloud.google.com/spanner/docs/schema-and-data-model

[38] V. Tropashko, "Nested Intervals Tree Encoding in SQL," arXiv:cs/0402051, 2004.

[39] R. Fagin, A. Lotem, and M. Naor, "Optimal Aggregation Algorithms for Middleware," *J. Computer and System Sciences*, vol. 66, no. 4, pp. 614–656, 2003.